OTUS

Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Ставьте “+” в чат, если да
Напишите, если есть проблемы

Вводные

Тема вебинара

Функциональные элементы Kotlin

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
Телеграм

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Маршрут вебинара

- Разберемся с функциями,
- ... лямбдами,
- ... scope-функциями
- ... коллекциями
-
- Рефлексия

Цели вебинара

1. Начать писать первые программы на Kotlin

Функции в Kotlin

Функции в Котлин

Ключевое слово fun от “function”

```
fun main() {
    println("Hello World!")
}
```

```
fun main(vararg args: String) {
    println("Hello World with args $args!")
}
```

```
fun main(args: Array<String>) {
    println("Hello World with args $args!")
}
```

Вызов функции

В простом случае как во всех других языках

```
fun double(x: Int): Int {
    return x*x
}

val double = double(4)
```

Функции в Kotlin
Аргументы

Значения аргументов по умолчанию

Можно явно указывать значения аргументов в определении функции.
Задавать их при вызове необязательно

Перемешивать в определении аргументы со значениями и без не следует

```
fun read(
    b: ByteArray,
    off: Int = 0,
    len: Int = b.size,
) { /*...*/ }

read(byteArrayOf(0,1,2,3))
```

Именованные аргументы функции

При вызове можно явно указывать имена аргументов и располагать в произвольном порядке

Рекомендуется всегда именовать аргументы при их числе более двух-трех

- Читаемость повышается;
- Вероятность ошибок при изменении семантики снижается.

```
fun reformat(
    str: String,
    normalizeCase: Boolean = true,
    upperCaseFirstLetter: Boolean = true,
    divideByCamelHumps: Boolean = false,
    wordSeparator: Char = ' ',
) { /*...*/ }

reformat(
    "String!",
    false,
    upperCaseFirstLetter = false,
    divideByCamelHumps = true,
    '_'
)
```

Перемешивание видов аргументов

```
1  fun countNewSteps(currentStepCounter: Int, from: Int, to: Int = 2): Int {
2      // some code
3  }
4  stepCounter = countNewSteps(5, 10)
```

```
1  fun countNewSteps(currentStepCounter: Int, from: Int = 10, to: Int): Int {
2      // some code
3  }
4  countNewSteps(5, 10) // compilation error <to> not passed
```

```
1  fun countNewSteps(currentStepCounter: Int, from: Int = 10, to: Int): Int {
2      // some code
3  }
4  countNewSteps(5, to = 10) // it's ok
5  countNewSteps(
6      currentStepCounter = 5,
7      from = 10,
8      to = 10
9  ) // now it works like swiss watch
```

Vararg аргументы

vararg args: String ~ args: Array<String>

Отличия:

<u>args: Array<String> =></u>
main(arrayOf(“One”, “Two”, “Three”))

<u>vararg args: String =></u>
main(“One”, “Two”, “Three”)
Или
main(*arrayOf(“One”, “Two”, “Three”))
Или
main(args = arrayOf(“One”, “Two”, “Three”))

```
fun main() {
    println("Hello World!")
}

fun main(vararg args: String) {
    println("Hello World with args $args!")
}

fun main(args: Array<String>) {
    println("Hello World with args $args!")
}
```

Функции в Kotlin
Результат функции

Возврат результата

Для возврата результата необходимо указать его тип
Возврат с помощью return

В сокращенной семантике фигурные скобки не нужны, тип возвращаемого результата опциональный

```
fun double(x: Int): Int {
    return x*x
}

val double = double(4)

fun triple(x: Int): Int = x*x*x

fun withExpression(x: Int): Int = when(x) {
    0 -> 100
    10 -> 1
    else -> 0
}
```

Необязательно

Тип Unit

Unit - Тип “Ничто” (Пусто)

```
1  public object Unit {
2      override fun toString() = "kotlin.Unit"
3  }
```

```
// 1
fun wakeUp(): Unit { // явно ничего не возвращаем
    print("I'm not sleeping")
}
// 2
fun wakeUp() { // неявно fun wakeUp(): Unit
    print("I'm not sleeping")
}

// 3
fun wakeUp(): Any = print("I'm not sleeping") // неявно `Unit: Any` inheritance

wakeUp() // prints "I'm not sleeping"
```

Тип Nothing

```
1  public class Nothing private constructor()
```

Nothing - Тип “Совсем Ничто” (Без завершения)

Предназначен для корректного вычисления типов

```
1  fun someFun() {
2      while(true) {
3          continue
4      }
5  } // It's not Unit -> Nothing
```

```
1  public inline fun TODO(): Nothing = throw NotImplementedError()
2
3  fun calculateMultiplication(first: Int, second: Int): Int = TODO()
```

```
val map = mapOf("key" to "text")

fun reportError(): Nothing = throw RuntimeException("Error happened") // we must set to Nothing,
because not UNIT

val data: String = map["key"] ?: reportError() // Compiles! Note the type is String.
```

Unit vs Nothing

Типы важны для вычисления типа результат

Unit требует чтобы был возвращен
результат именно Unit

Nothing совместим с любым типом

```
fun els(): Nothing = throw Exception("")
fun x(x: Int) : Any = when(x) {
    0 -> "1"
    1 -> println("xx")
    else -> els()
}
```

Conditional branch result of type Unit is implicitly cast to Any

```
fun els(): Nothing = throw Exception("")
fun x(x: Int) : Unit = when(x) {
//    0 -> "1"
    1 -> println("xx")
    else -> els()
}
```

Функции в Kotlin
Генерики

Обобщенные типы

Когда логика не зависит от типа, можем использовать генерики

```
fun <T> oneElementList(arg: T): List<T> = listOf(arg)
val l1: List<String> = oneElementList("str")
val l2: List<Int> = oneElementList(1)
val l3: List<Long> = oneElementList(1L)
```

Обобщенные типы с ограничениями

Можем ограничивать допустимые типы генериков

```
fun <T> oneElementList(arg: T): List<T> = listOf(arg)
val l1: List<String?> = oneElementList( arg: null)
fun <T: Any> oneElementListN(arg: T): List<T> = listOf(arg)
val l2: List<String?> = oneElementListN( arg: null)
```

Null can not be a value of a non-null type TypeVariable(T)

Change parameter 'arg' type of function 'oneElementListN' to 'Any'

Потеря типов в рантайме

Контроль типов в генериках выполняется только во время компиляции

В рантайме типы не могут быть проконтролированы

Kotlin старается спасти типы до последнего на уровне компилятора

```
class X {
    @Test
    fun x() {
        var any: Any = 1
        typedGeneric(any)
        // Prints "Int"
        any = "str"
        typedGeneric(any)
        // Prints "String"
        val lst: List<Int> = listOf(1)
        typedGeneric(lst)
        // Prints "List<*>"
    }
}

fun <T: Any> typedGeneric(arg: T) {
    println("type: ${arg::class.simpleName} of T: ${T::class}")
    when (arg) {
        is Int -> println("Int")
        is String -> println("String")
        is List<*> -> println("List<*>")
        is List<Int> -> println("List<Int>")
        else -> println("Other type")
    }
}
```

Функции в Kotlin
inline

Модификатор inline

```
“Виртуальная” функция, существует только
  в коде, при компиляции тело дублируется

Назначение:
– Ускорение в критических участках кода
– Доступ к локальной среде исполнения
  (корутины и пр.)
– Доступ к типам через reified
```

```
inline fun some(x: Int) = x*x
```

Модификатор reified

Позволяет преодолеть проблему потери типа

```
inline fun <reified T: Any> typedGeneric(arg: T) {
    println(
        "type: ${arg::class.simpleName} " +
            "of T: ${T::class}"
    )
}
// ————————————————————————————————————-
inline fun <reified T: Any> typedGeneric(arg: T) =
    typedGeneric(arg, T::class)

fun <T: Any> typedGeneric(arg: T, clazz: KClass<T>) {
    println(
        "type: ${arg::class.simpleName} " +
            "of T: $clazz"
    )
}
```

Функции в Kotlin
tailrec

Модификатор tailrec

```
Обеспечивает рекурсивный вызов функции
  без использования стека
Рекурсивный вызов преобразуется в цикл
```

```
tailrec fun findFixPoint(x: Double = 1.0): Double =
    if (abs(x - cos(x)) < eps) x else findFixPoint(cos(x))


tailrec fun fibonacci(n: Int, a: Int = 0, b: Int = 1): Int =
    when (n) {
        0 -> a
        1 -> b
        else -> fibonacci(n - 1, b, a + b)
    }
```

Лямбда функции в Kotlin

Виды функций

```
// simple function
fun function() = print("hello, otus!")

// anonymous function, Unit can be omitted
val iAmFunction = fun(): Unit = print("hello, otus!")

// simple lambda
val iAmLambda = { print ("hello") }
iAmLambda() // prints hello

// with argument
val giveName = { str: String -> "My name is $str"}

print(giveName("Dee")) // prints My name is Dee

// with type
val multiplication: (Int, Int) -> String = {first: Int, second: Int -> "Multiplication = ${first *
second}"} // we can omit type

multiplication(1, 4) // prints Multiplication = 4
```

Анонимные функции

```
// anonymous function, Unit can be omitted

val iAmFunction = fun(): Unit {
    print("hello, otus!")
}
val x = iAmFunction()
```

Лямбда-функции

(String) -> Unit – это тип лямбды

it – имя аргумента по умолчанию, если не задано явно

```
typealias MyLambda = (String) -> Unit
val x3: MyLambda = { println("hello $it") }
```

```
val iAmLambda = {
    print ("hello")
}
val x = iAmLambda() // prints hello

val iAmLambda1 = { arg: String ->
    print ("hello $arg")
}
val x1 = iAmLambda1("Dennis") // prints hello Dennis

val iAmLambda2: (String) -> Unit = {
    print ("hello $it")
}
val x2 = iAmLambda1("Dennis") // prints hello Dennis
```

Функции высшего порядка

High Order Functions

Принимают в качестве аргумента другие функции

Если функциональный аргумент последний, то лямбду можно вынести за ()

```
fun strengthCalculator(
    strength: Int,
    calculator: (Int) -> Int,
): Int = calculator(strength) /100

val myNewStrength = strengthCalculator(1000, {
    it * it
}) // 10000

val myNewStrength = strengthCalculator(1000) {
    it * it
} // 10000
```

Функции высшего порядка

High Order Functions

Принимают в качестве аргумента другие функции

Если функциональный аргумент последний, то лямбду можно вынести за ()

```
fun echo(
    sayIt: (toPerson: String, withAge: Int) -> String,
    name: String,
    age: Int,
) {
    sayIt(name, age)
}

echo({ toPerson, _ ->
    "Hello, $toPerson"
}, "Alice", 33)
```

Расширения

Extensions

Замена для Helper/Utility classes

```
1   fun Int.toCustomString() = "This is integer $this" // this refers to Int object
2   fun Double.toCustomString() = "This is double"
3   fun Int.toLong(): String = "Convert int to Long"
4   fun Any?.toCustomString(): String {
5       return if (this == null) {
6           "Nothing to return"
7       } else {
8           "$this this is custom string"
9       }
10  }
11
12  var integer: Int? = null
13  val number = 10
14  print(10.toCustomString()) // "This is integer 10"
15  print(number.toLong()) // Prints 10L not Convert int to Long
16  val double = 10.3
17  print(double.toCustomString()) // This is double
18  print(integer.toCustomString()) // Prints Nothing to return, 'cuz it's null
```

Ресиверы лямбды

Лямбды могут быть объявлены в виде “расширений”:

A.(B) -> C

```
val greet: String.() -> Unit = { println("Hello, $this")}

greet("Dee")
"CoolGuy".greet()


val greetWithSurname: String.(String) -> Unit = { surname: String ->
println("Hello, $this $surname")}

greetWithSurname("Ivan", "Gorky")
"Ivan".greetWithSurname("Gorky")
```

Инфиксные функции

Инфиксные функции допускают вызов без любых скобок

Являются элементом DSL

Требования:

- Ровно один аргумент
- Является расширением
- Ключевое слово infix

```
infix fun String.withNum(num: Int) = "$this with ($num)"
val res = "My string" withNum 22
// Prints "My string (22)"
```

ПОВТОРИМ ИЗУЧЕННОЕ

Повторение - мать учения

```
1. val x = 2 power 3 // Напишите реализацию функции power

2. val x = 2.0 power 3 // Напишите реализацию функции power для любых чисел

3. val y = func("string") { // Напишите реализацию функции func
   "perfix: $it"
} // "lambda (prefix: string)
```

Функции области видимости

Scope functions

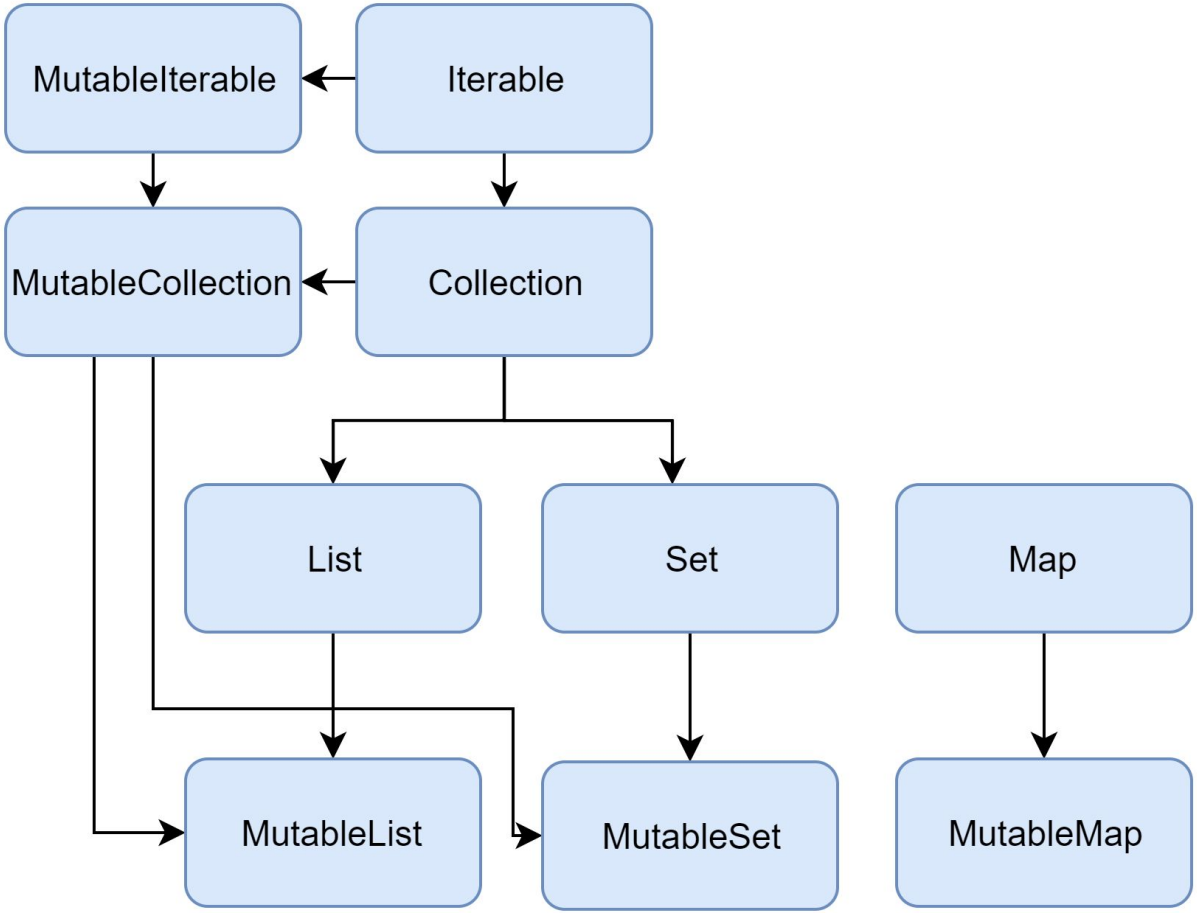
```
assertEquals("4456", with("445") { this + "6" })
assertEquals("445", "445".apply { this + "6" })
assertEquals("445", "445".also { it + "6" })
assertEquals("4456", "445".run { this + "6" })
assertEquals("4456", "445".let { it + "6" })
```

https://kotlinlang.org/docs/scope-functions.html

```
inline fun <T, R> with(receiver: T, block: T.()
-> R): R {
    return receiver.block()
}

inline fun <T> T.also(block: (T) -> Unit): T {
    block(this)
    return this
}

inline fun <T> T.apply(block: T.() -> Unit): T
{
    block()
    return this
}

inline fun <T, R> T.let(block: (T) -> R): R {
    return block(this)
}

inline fun <T, R> T.run(block: T.() -> R): R {
    return block()
}
```

Коллекции

Коллекции


MutableIterable
Iterable
MutableCollection
Collection
List
Set
Map
MutableList
MutableSet
MutableMap

Коллекции

List<T>, MutableList<T> - listOf(), mutableListOf()

Map<K,V>, MutableMap<K,V> - mapOf(), mutableMapOf()

IntRange, LongRange, CharRange

IntProgression, LongProgression, CharProgression

Sequence<T>, IntSequence, LongSequence - sequenceOf()

Flow<T>, IntFlow, LongFlow - flowOf()

```
val l1 = listOf(1,2,3)
val l2 = mutableListOf(1,2,3)

val m1 = mapOf(
    1 to "str 1",
    2 to "str 2"
)
val m2 = mutableMapOf(
    1 to "str 1",
    2 to "str 2"
)

val r1: IntRange = 1..10
val p1: IntProgression = 100 downTo 10 step 2
val p2: IntProgression = 1 until 10 step 3
```

Операции с коллекциями

- Transformations
  - map, mapIndexed, mapNotNull, mapIndexedNotNull
  - zip
  - associateWith, associateBy
  - flatten, flatMap
  - joinToString
- Filtering
- plus and minus operators
- Grouping
- Retrieving collection parts
- Retrieving single elements
- Ordering
- Aggregate operations

```
val res = (0..10)
    .associateBy {
        it
    }
    .map { it.key to it.value*2 }
    .filter { (it.second % 3) != 0 }
    .joinToString("\n") {
        "${it.first} = ${it.second}"
    }
```

Рефлексия

Цели вебинара

Проверка достижения целей

1. Начать писать первые программы на Kotlin

Список материалов для изучения

- https://www.jetbrains.com/
- https://ru.wikipedia.org/wiki/JetBrains
- https://kotlinlang.org/
- https://gradle.org/
- https://github.com/otuskotlin/202505-ok-marketplace
- https://github.com/crowdproj
- https://medium.com/@fatihcoskun/kotlin-scoping-functions-apply-vs-with-let-also-run-816e4efb75f5

Рефлексия

С какими основными мыслями
и инсайтами уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Следующий вебинар

Модуль 1: Введение в Kotlin

Объектно-ориентированное программирование

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание! Приходите на следующие вебинары